



UNIVERSIDADE
FEDERAL DO CEARÁ
CAMPUS QUIXADÁ

Gerência de Configuração

Gerenciamento de Releases

Profa. Lana Mesquita



Conteúdo da aula

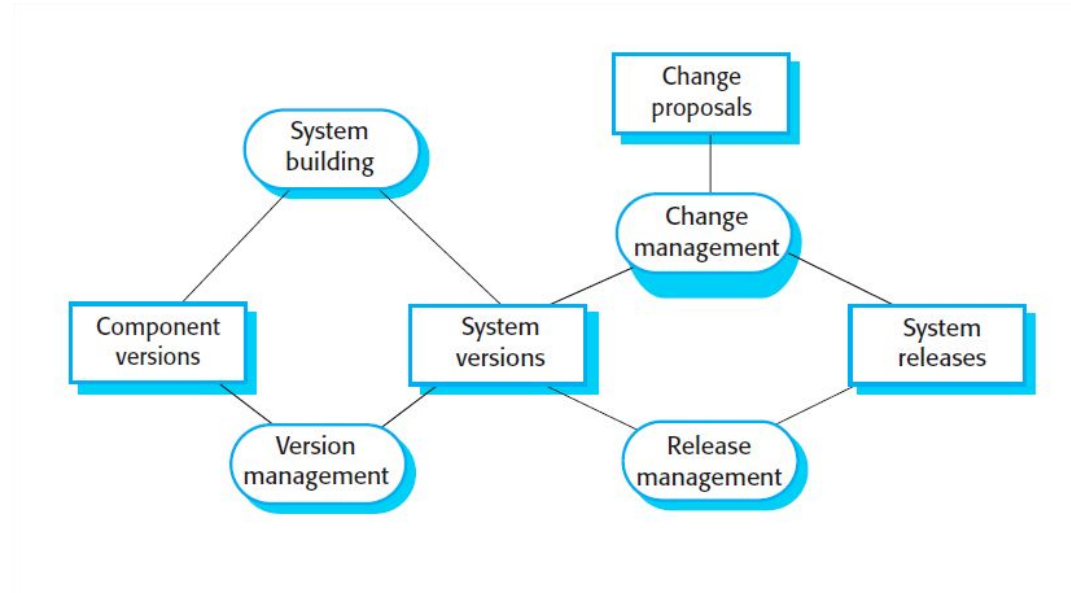
- GC
- Releasing
- Versionamento Semântico
- Software release life cycle
- Padrões de Branches
- Branching policies
 - GitFlow
 - Trunk-Based
- git tag
 - tags e releases no GitHub

Revisitar conteúdos de Práticas de GC

https://docs.google.com/presentation/d/1QdPKIcJzHoJFnoENWo9QmMpJe52KWh6r-LfZZGyi_wc/edit?slide=id.g37a0b7b0a00_0_178#slide=id.g37a0b7b0a00_0_178

Gerência de Configuração

- Atividades de Gerência de configuração



Gerência de Configuração

- **Controle de versão:** envolve acompanhar as múltiplas versões dos componentes do sistema e garantir que as alterações feitas nos componentes por diferentes desenvolvedores não interfiram entre si.
- **Gerenciamento de release:** envolve a preparação do software para liberação externa e a manutenção do controle das versões do sistema que deve ser liberada para uso (produção).

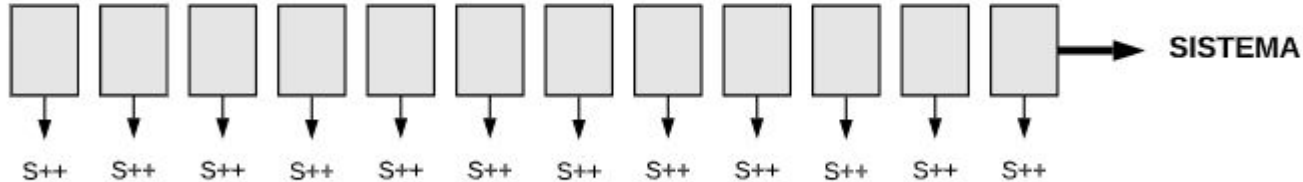
Controle de versão

O controle de versão depende do processo de desenvolvimento adotado e das práticas aplicadas (CI ou CD por exemplo).

As próximas figuras comparam desenvolvimento em Waterfall e Ágil.

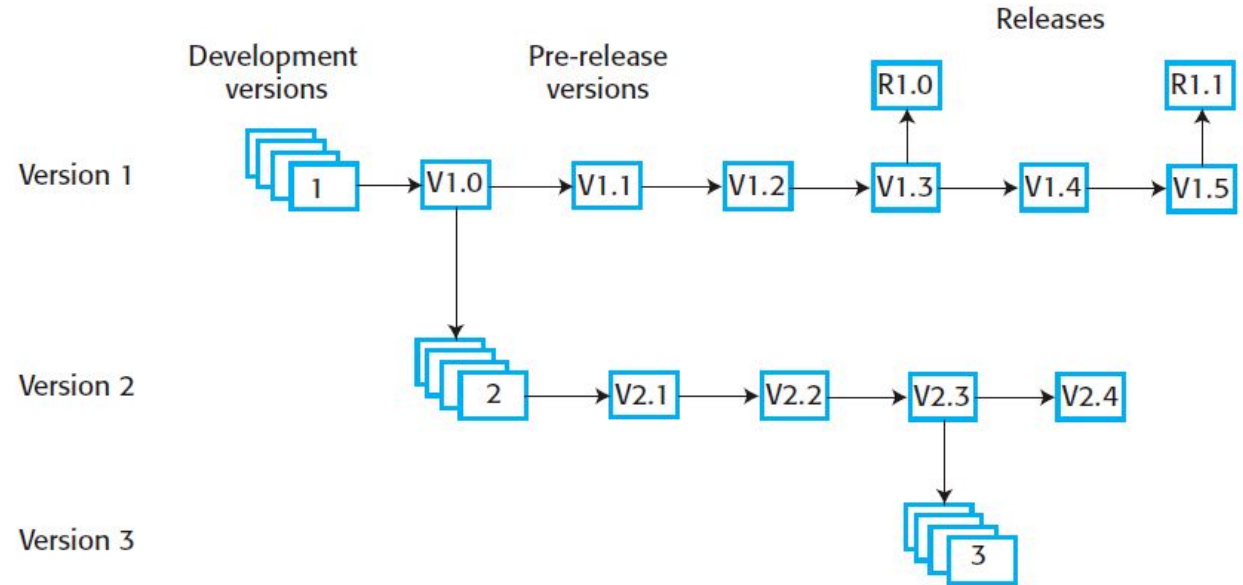


Desenvolvimento usando um Processo Waterfall. O sistema fica pronto apenas no final.



Desenvolvimento usando um Processo Ágil. A cada iteração (representada pelos retângulos) gera-se um incremento no sistema (S++), que já pode ser validado e testado pelos usuários finais.

Múltiplas versões



Releasing / Lançamento

processo manual

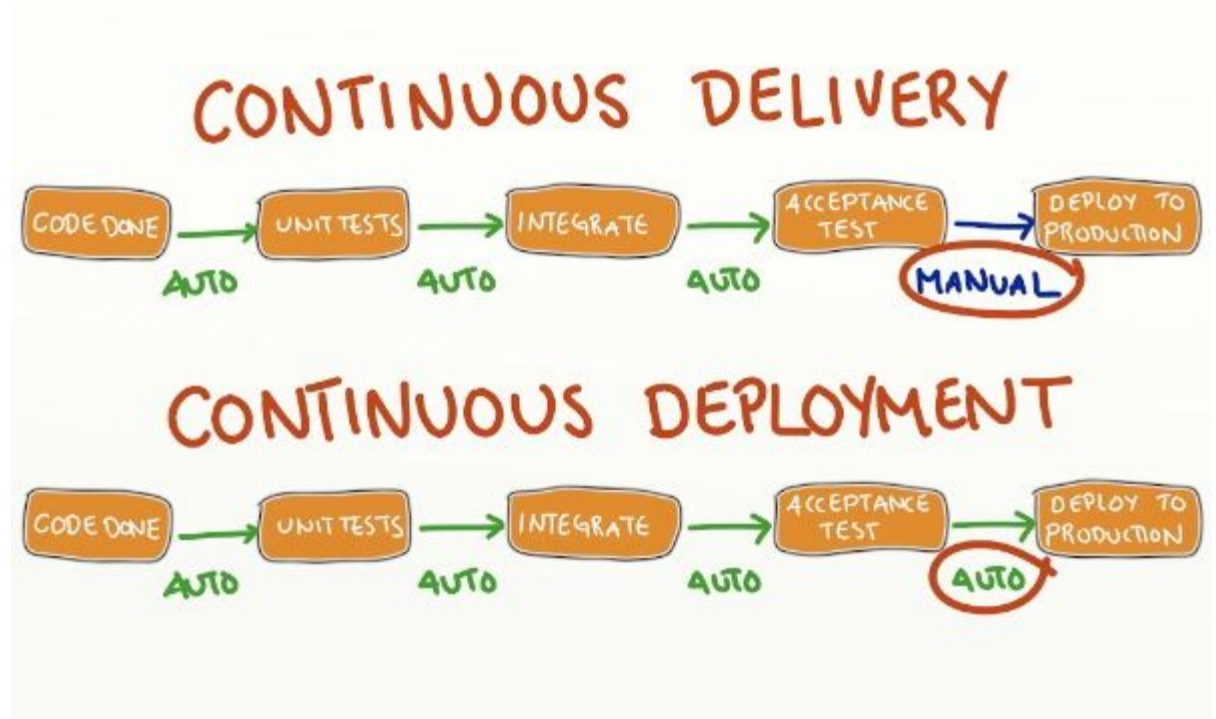
A criação de lançamento é o processo de criação da coleta de arquivos e documentação.

Este processo envolve várias etapas:

1. Identificar o código executável dos programas e de todos os arquivos de dados associados no sistema de controle de versão e marcar com o identificador de versão (**tag**).
2. Preparar configurações específicas para diferentes sistemas operacionais e hardware.
3. Atualizar instruções de configuração do sistema para clientes.
4. Preparar os scripts de instalação.
5. Publicar a versão e sua descrição em um link de documentação do sistema acessível.
6. Após disponibilizar todas as informações, entregar uma imagem executável do software para ser distribuída aos clientes/usuários.

Releasing / Lançamento

processo
automatizado
com CD



Versionamento Semântico

Versionamento Semântico

- Se as especificações das dependências são muito especificadas você corre o risco de um bloqueio de versão.
 - Tipo atualizar algo pequeno e ter que gerar toda uma nova versão impactando em sistemas integrados ou outras partes do software.
- Se as dependências são vagamente especificadas, você irá inevitavelmente ser mordido pela ‘promiscuidade da versão’ (assumindo compatibilidade com futuras versões mais do que é razoável).
- De autoria de Tom Preston-Werner, criador do Gravatars e co-fundador do GitHub, o versionamento semântico se propõe como solução deste problema com uma regra bem simples para versionar sistemas.

Especificação da Semântica de Versionamento

Foco em APIs.



- Versão Maior (MAJOR): quando fizer mudanças incompatíveis na API.
- Versão Menor (MINOR): quando adicionar funcionalidades mantendo compatibilidade.
- Versão de Correção (PATCH): quando corrigir falhas mantendo compatibilidade.
- Pre-Release – Versão candidata: Esta versão pode não satisfazer os parâmetros de compatibilidade.

Rótulos adicionais para pré-lançamento (pre-release) e metadados de construção (build) estão disponíveis como extensão ao formato MAJOR.MINOR.PATCH.

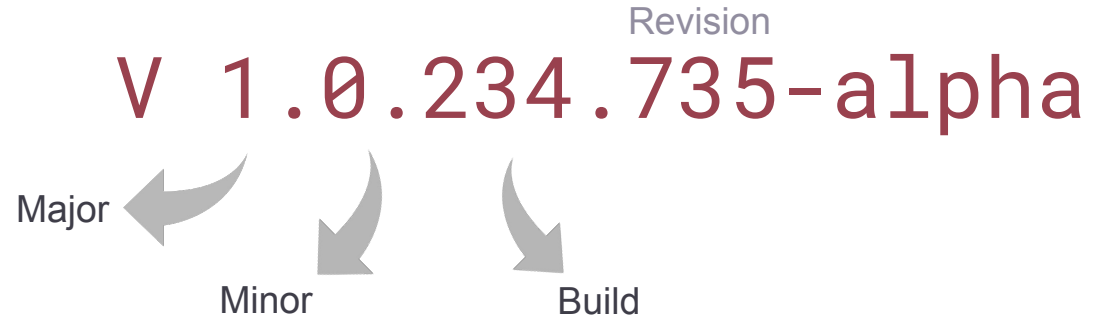
Especificação da Semântica de Versionamento

Regras

- Um número de versão normal DEVE ter o formato de X.Y.Z, onde X, Y, e Z são inteiros não negativos, e NÃO DEVE conter zeros à esquerda.
- Cada elemento DEVE aumentar numericamente. Por exemplo: 1.9.0 -> 1.10.0 -> 1.11.0.
- Uma vez que um pacote versionado foi lançado (released), o conteúdo desta versão **NÃO DEVE ser modificado. Qualquer modificação DEVE ser lançado como uma nova versão.**
- No início do desenvolvimento, a versão Maior DEVE ser zero (0.y.z). A API pública NÃO DEVE ser considerada estável.
- O software em produção deve iniciar em 1.0.0
- É indicado que a versão inicial seja a 0.1.0.

Versionamento (Microsoft)

Versionamento para Microsoft



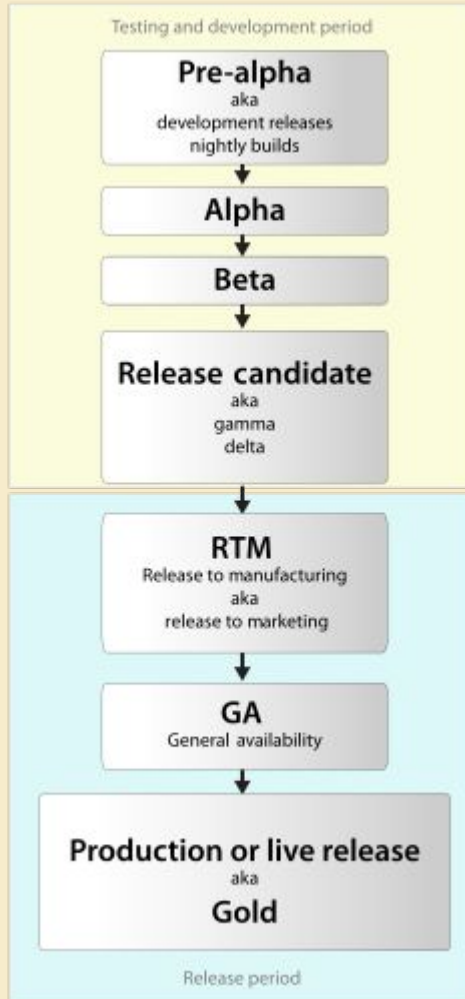
Versionamento para Microsoft

Revision
V 1.0.234.735
Major
Minor
Build

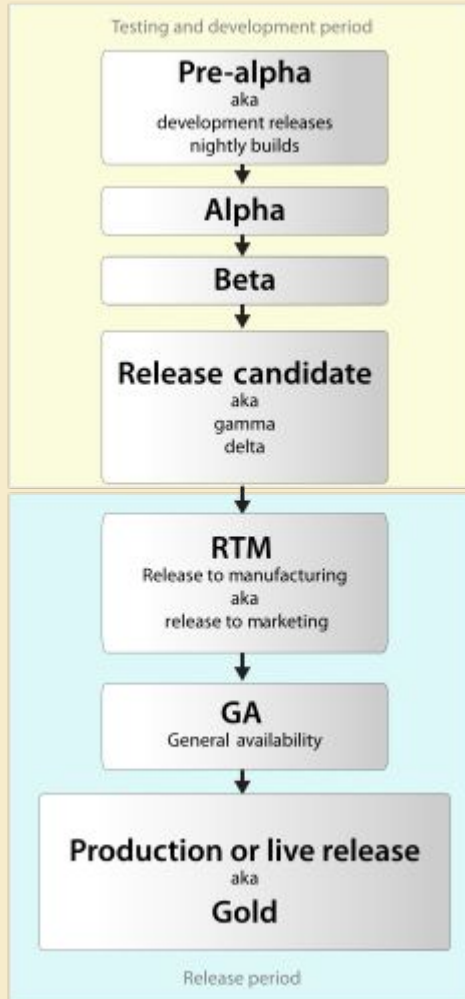
- **Major:** um número de versão pode indicar uma reescrita importante de um produto onde a compatibilidade retrógrada não pode ser assumida.
- **Minor:** um aprimoramento significativo com a intenção de retrocompatibilidade. Este número de versão Minor pode indicar uma versão pontual de um produto ou uma nova versão totalmente compatível com versões anteriores.
- **Build:** Uma diferença no número de compilação representa uma recompilação do mesmo código. Pode indicar mudanças de processador, plataforma ou compilador.
- **Revision:** os números de revisão dentro de um mesmo conjunto de Major e Minor devem ser totalmente intercambiáveis. Um número de revisão mais alto pode ser usado em uma compilação que corrige uma falha de segurança em um conjunto previamente lançado.

Versões subsequentes que diferem apenas pelos números de compilação (build) ou revisão são consideradas atualizações de Hotfix da versão anterior.

Software release life cycle

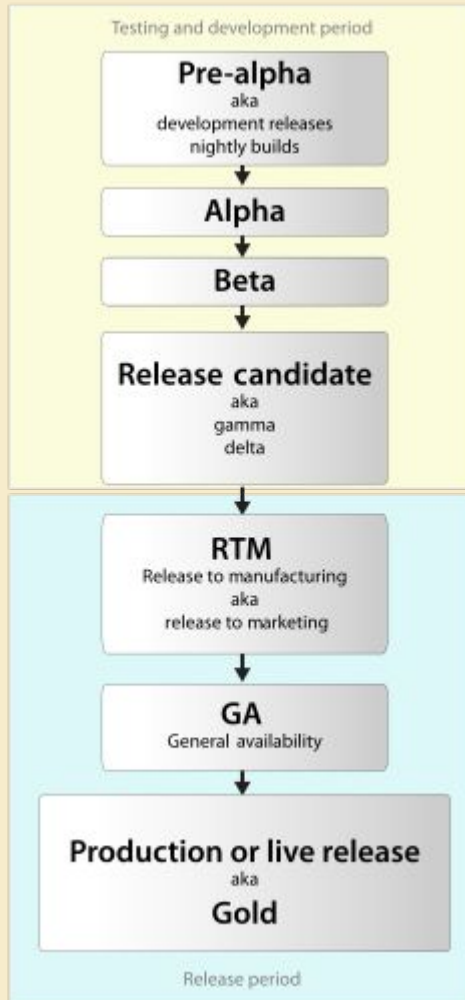


- **Pre-alpha:** atividades realizadas durante o projeto de software antes dos testes formais, como análise de requisitos, design de software, desenvolvimento de software e testes de unidades.
- **Alpha:** é a primeira fase de testes de software. Nesta fase, os desenvolvedores geralmente testam o software usando técnicas de caixa branca, para posteriormente equipes de teste realizarem testes de caixa-preta.
 - Um software Alpha pode conter erros graves e instabilidades podem causar falhas ou perda de dados.
 - O software Alpha pode não conter todos os recursos que estão planejados para a versão final.
 - Normalmente essa versão não é disponibilizada externamente, com exceção de softwares OpenSource.
 - A fase alfa geralmente termina com um congelamento de recursos, indicando que não serão adicionados mais recursos ao software.



- **Beta:** inicia geralmente quando o software está completo, recursos, mas contém bugs conhecidos ou desconhecidos.
 - O foco dos testes beta é reduzir os impactos aos usuários, muitas vezes incorporando testes de usabilidade.
 - O processo de entrega de uma versão beta para os usuários normalmente é quando o software fica disponível fora da organização que o desenvolveu.
 - As versões beta de software podem ser públicas ou privadas, podendo ser abertas ou apenas disponíveis para um público limitado.

Testadores beta tendem a oferecer seus serviços gratuitamente, mas muitas vezes recebem versões do produto que testam, descontos na versão de lançamento ou outros incentivos.
 - Caso Gmail -> Perpetual Beta (2004 – 2009)



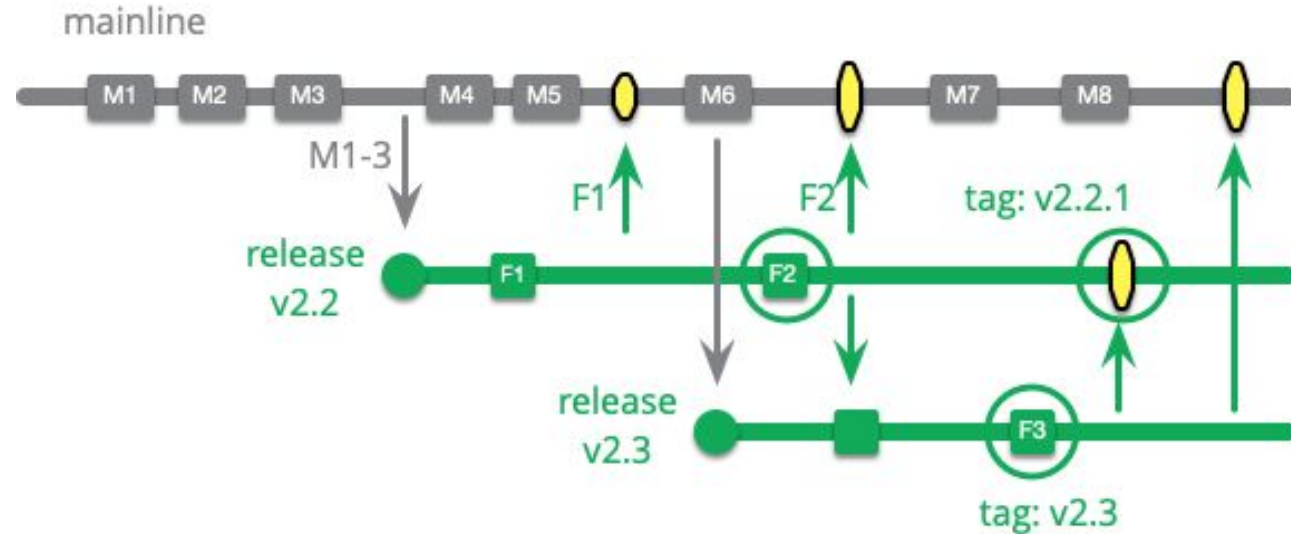
- **Release to manufacturing (RTM)** é um termo usado quando um produto de software está pronto para ser entregue.
- **Disponibilidade geral (GA)** é o estágio em que todas as atividades de comercialização necessárias foram concluídas e um produto de software está disponível para compra.

Padrões de Branches



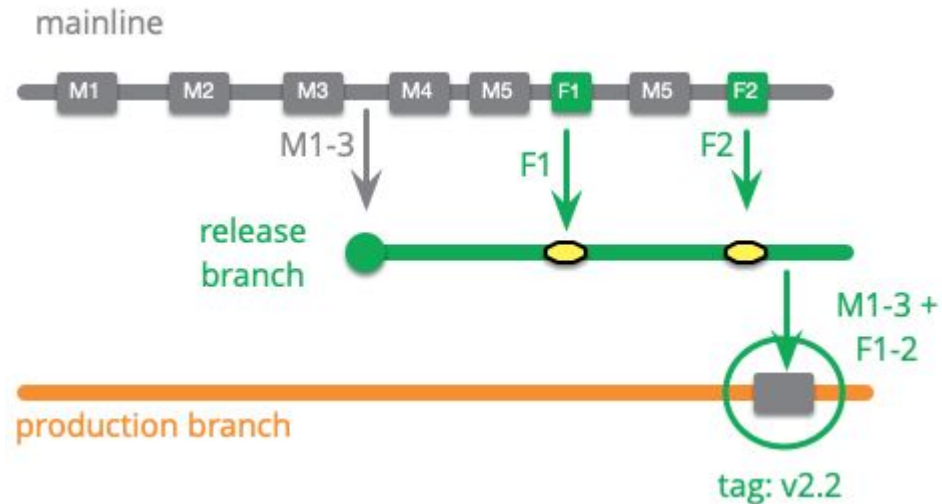
Padrões de Branches

Release Branch: são uma boa estratégia quando a equipe não consegue manter sua branch base em um estado saudável.



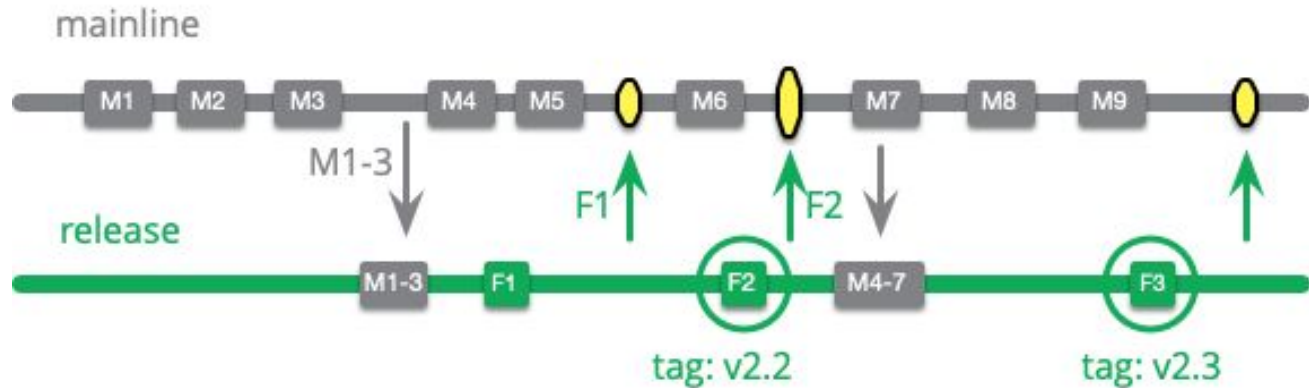
Padrões de Branches

Maturity Branch: uma branch que marca a versão mais recente de um nível de maturidade da base de código.



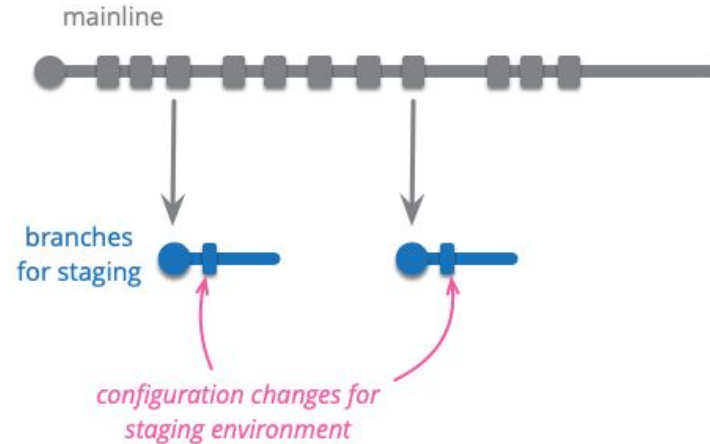
Padrões de Branches

Long Lived Release Branch: uma branch que marca a versão mais recente de um nível de maturidade da base de código.



Padrões de Branches

Environment Branch: é uma branch que contém commits para reconfigurar o produto para rodar em ambientes diferentes, exemplo: teste, staging ou produção.



Padrões de Branches

De acordo com Fowler, essa é a melhor escolha.

Release-Ready Mainline: mantém a linha base sempre saudável para que possa sempre ser colocada em produção

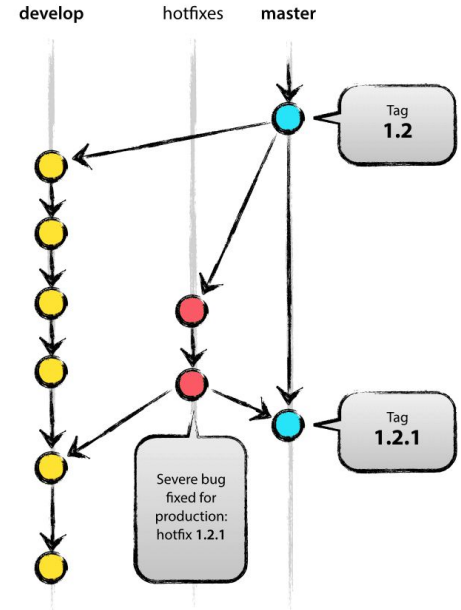


Branching Policy

Git Flow e Github Flow

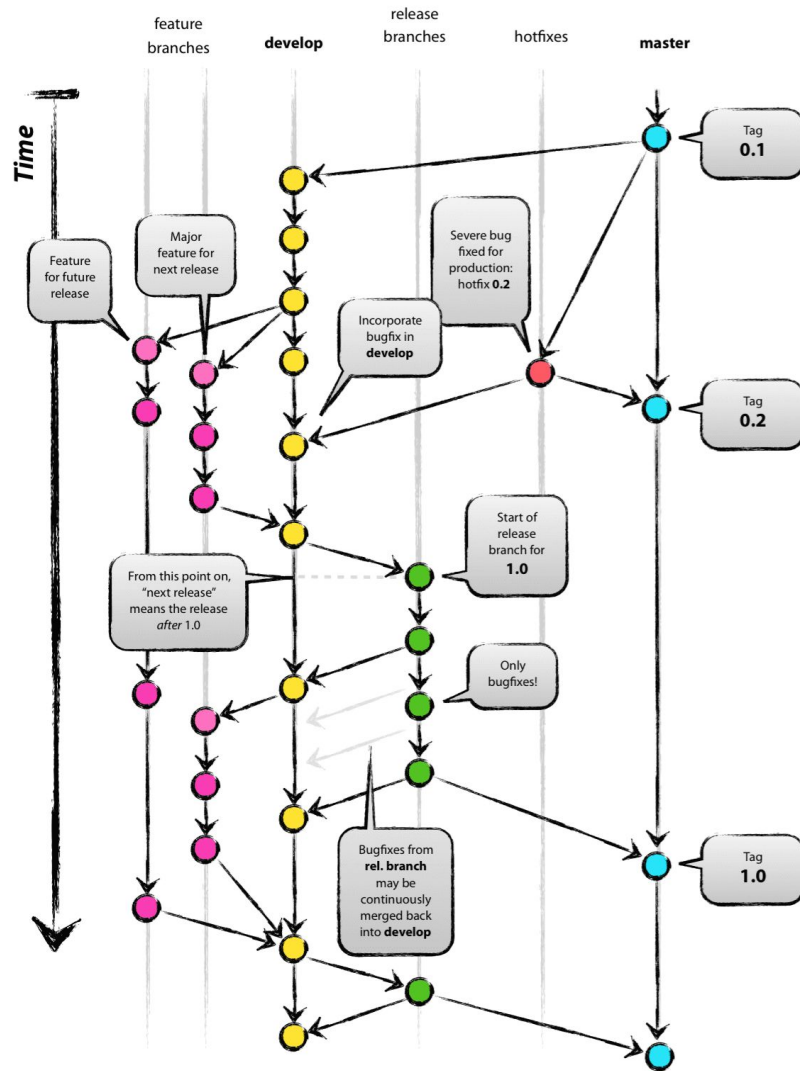
Git-flow

O Git-flow se tornou uma das políticas de ramificação mais comuns. Foi escrito por Vincent Driessen em 2010, surgindo quando o Git estava se tornando popular.



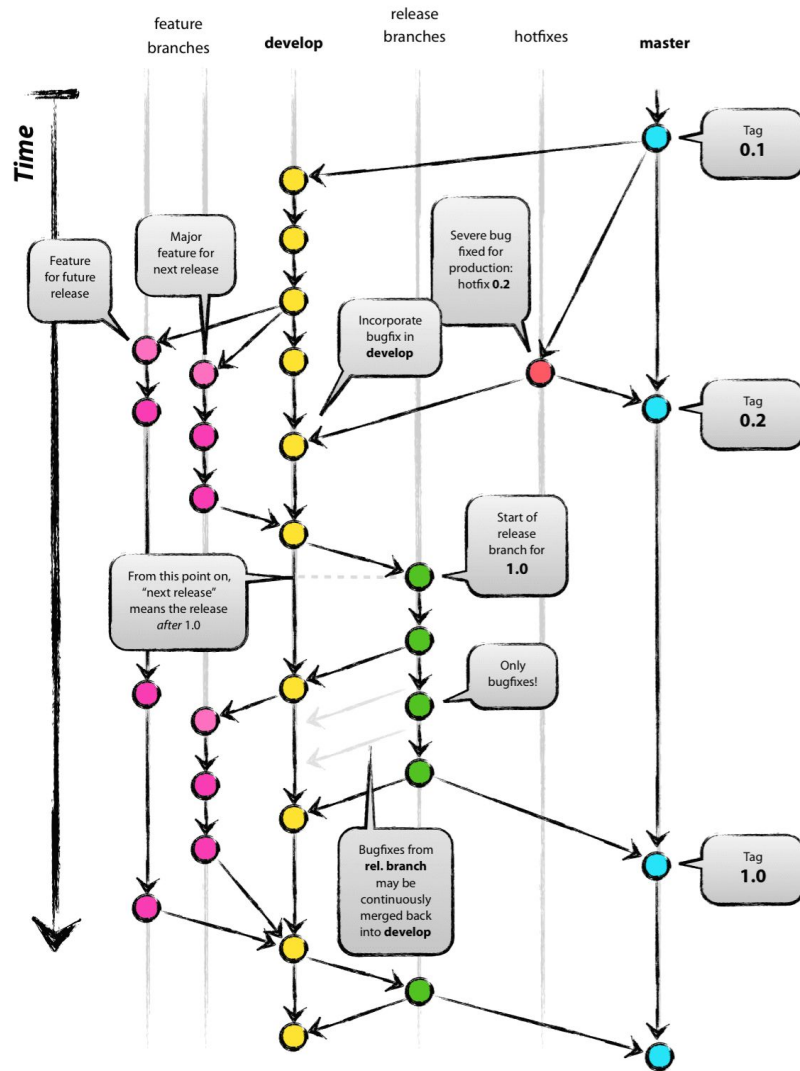
Git-flow

- A branch **Mainline** (*develop*) em um único repositório “origin”.
- Usa **Feature Branching** para coordenar múltiplos desenvolvedores.
- Desenvolvedores são encorajados a subir **Collaboration Branches** para coordenar com outros desenvolvedores trabalhos similares.
- A branch principal é chamada de **master**, usada como **Production Maturity Branch**.
- Git-Flow usa uma **Release Branch** para que o trabalho passe da *develop* para a branch de release **master**.
- Hotfixes são organizados numa **Hotfix Branch**.



A **master** irá conter todo código já testado, versionado que será entregue ao cliente e a **develop** é onde todo fluxo de trabalho irá ocorrer antes de fazer o release versionado que será feito merge na **master**.

A **develop** deve sempre conter o código mais atual, onde as branches de features serão ramificadas tendo ela como base.



feature: para novas implementações

release: para finalizar o release e tags

hotfix: para resolver problema crítico em produção que não pode esperar novo release

Quando usar Git-flow?

O git-flow foi projetado para os tipos de projetos em que há **múltiplas versões lançadas em produção**, como softwares instalados em clientes.

No entanto, muitos usuários adotaram o git-flow no contexto de um único webapp de produção — nesse ponto, essa estrutura de ramificação se torna mais complicada do que o necessário.

Embora o Git-Flow seja muito popular, é comum encontrar pessoas que dizem usá-lo, mas que, na verdade, fazem algo bem diferente. Muitas vezes, a abordagem real delas se aproxima mais do GitHub Flow.

Fluxo do GitHub Flow

diferente do Git-flow

Não tem branch de release.

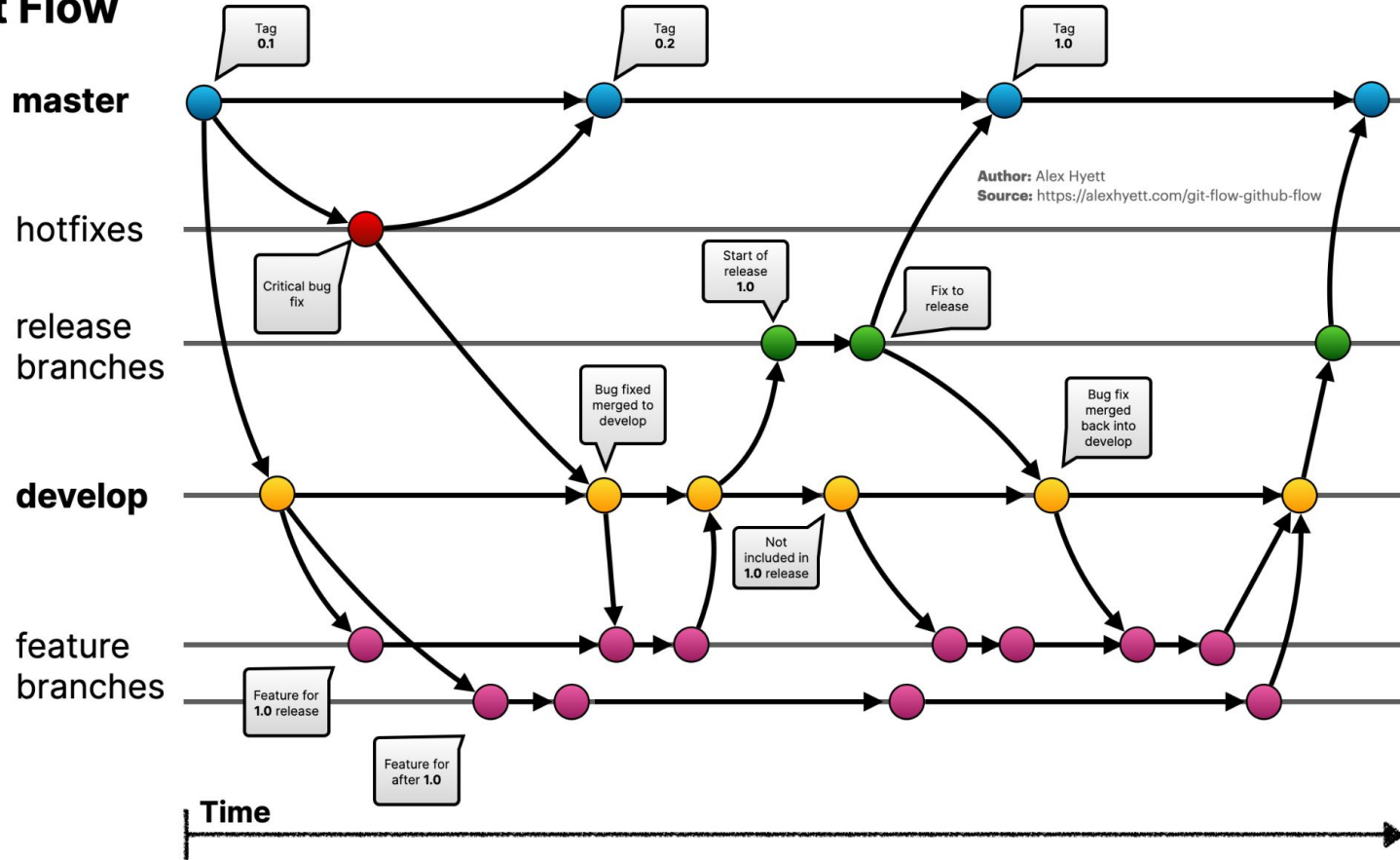
Não tem branch de hotfix, usa a feature mesmo.

Enviam seus branches de feature regularmente para o repositório central, para dar suporte à visibilidade, mas não há integração com a linha principal até que o recurso esteja concluído.

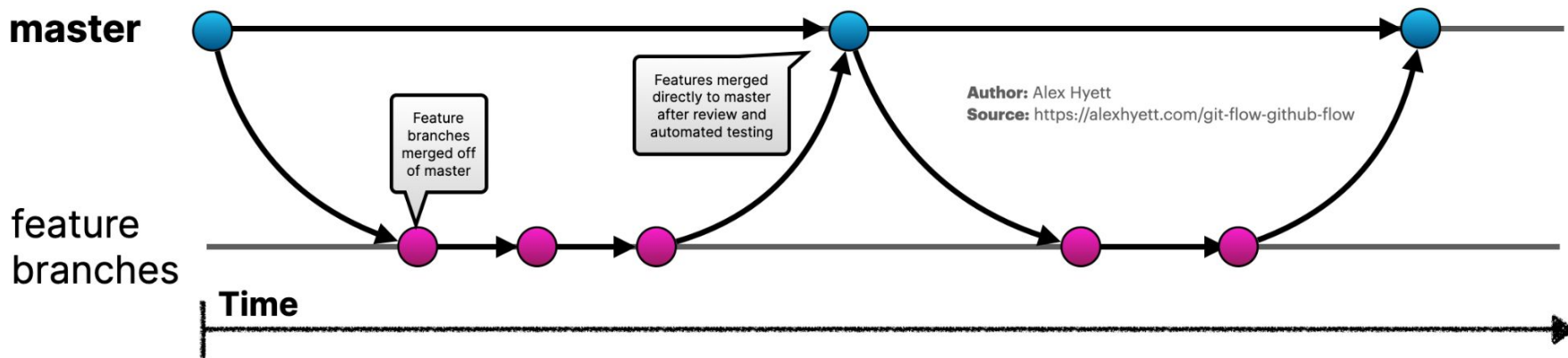
Trabalham com Pull Requests para solicitar o merge.

<https://docs.github.com/en/get-started/using-github/github-flow>

Git Flow



GitHub Flow



Trunk-Based Development

Trunk-based

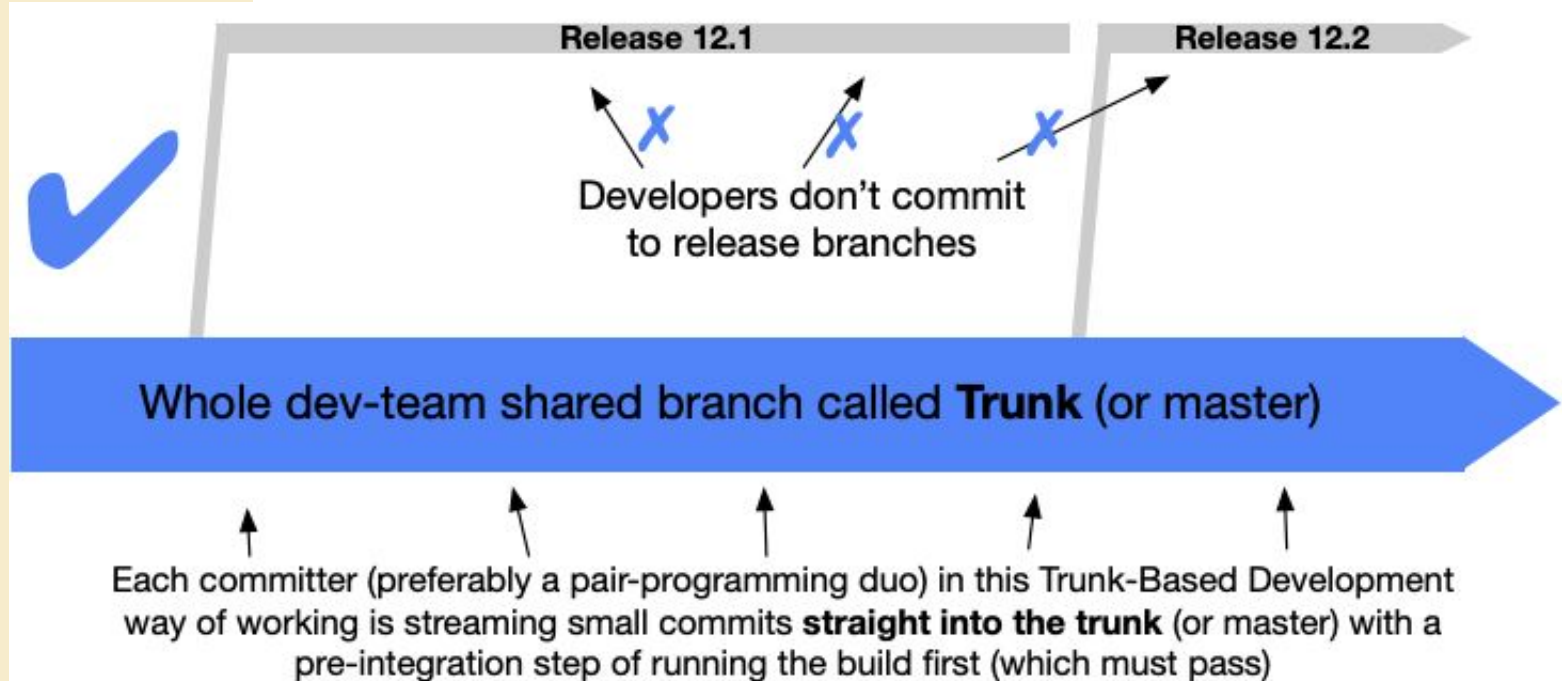
Alternativa ao Git Flow e Github Flow.

O Trunk-based se concentra em realizar todo o trabalho na branch ***trunk***, evitando qualquer tipo de ramificação de longa duração.

Equipes menores usam só uma branch ; equipes maiores podem usar a branches de ***feature*** de curta duração (não mais do que alguns dias – o que provavelmente equivale à Integração Contínua na prática).

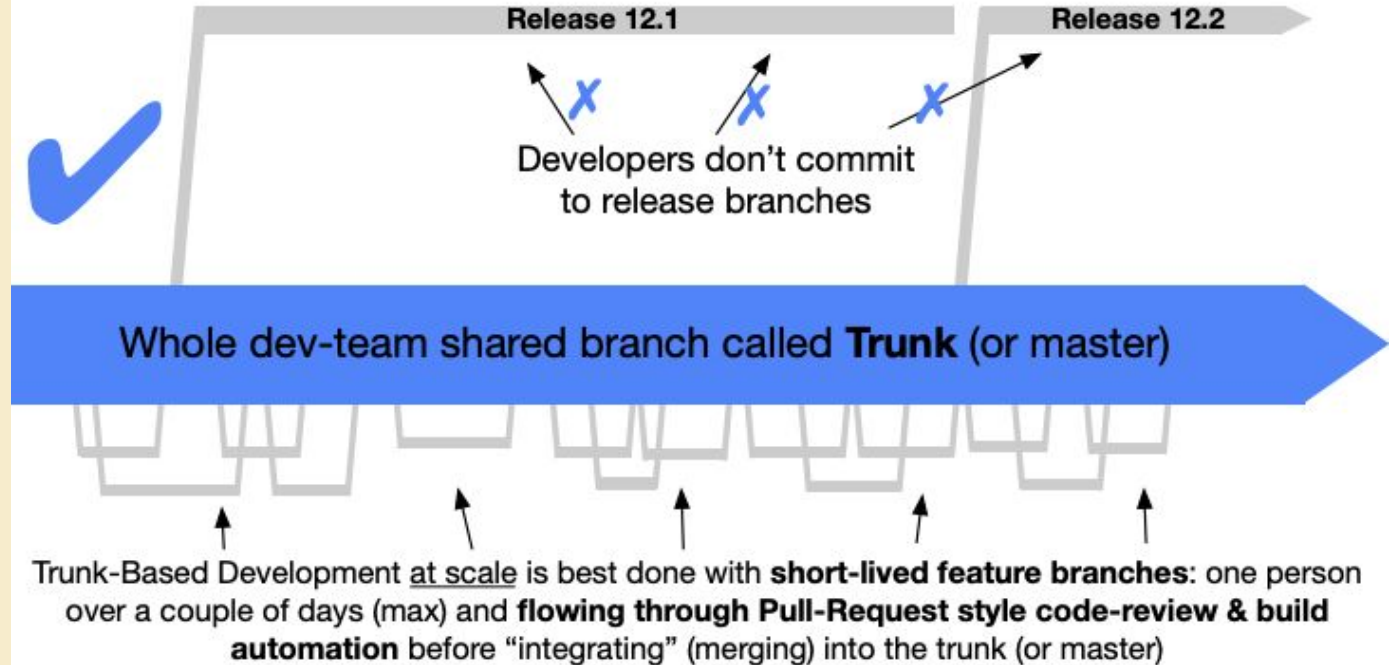
Equipes podem usar a branch ***release***

Trunk-based



Trunk-based

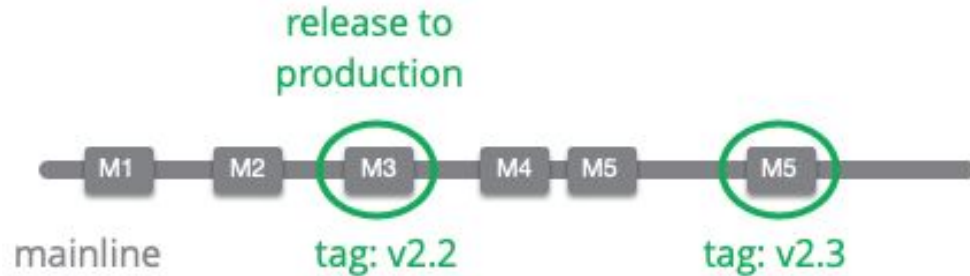
para times
maiores



git tag

Tag

- VCSs, como o Git, tem a habilidade de marcar pontos específicos na história como sendo importantes: **tags**, normalmente usadas para indicar pontos de releases.



git tag

git tag : Listar tags disponíveis

```
$ git tag v1.4-lw  
$ git tag  
v0.1  
v1.3  
v1.4  
v1.4-lw  
v1.5
```

```
$ git show v1.4-lw  
commit ca82a6dff817ec66f44342007202690a93763949  
Author: Scott Chacon <schacon@gee-mail.com>  
Date:   Mon Mar 17 21:52:11 2008 -0700
```

changed the version number

git tag

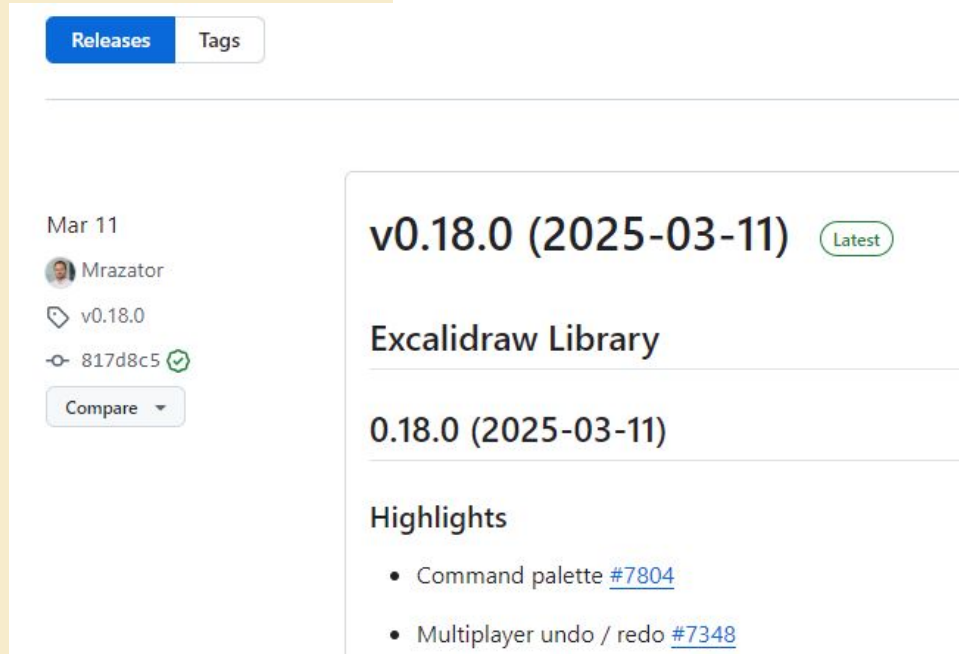
Para subir a tag para o repositório remoto use ***git push***.

```
$ git push origin v1.5
Counting objects: 14, done.
Delta compression using up to 8 threads.
Compressing objects: 100% (12/12), done.
Writing objects: 100% (14/14), 2.05 KiB | 0 bytes/s, done.
Total 14 (delta 3), reused 0 (delta 0)
To git@github.com:schacon/simplegit.git
 * [new tag]          v1.5 -> v1.5
```

Tag e Release no Github

Vamos ver na prática:

<https://github.com/excalidraw/excalidraw>



The screenshot shows the GitHub Releases page for the repository `excalidraw/excalidraw`. The "Releases" tab is selected. The latest release, `v0.18.0`, is highlighted with a green "Latest" badge and shows a date of `(2025-03-11)`. The release title is "Excalidraw Library". Below the title, the version `0.18.0` is repeated with the same date. The "Highlights" section lists two features: "Command palette" (linked to issue [#7804](#)) and "Multiplayer undo / redo" (linked to issue [#7348](#)). On the left sidebar, the release date "Mar 11" is shown, along with the user "Mrzator", the version "v0.18.0", the commit hash "817d8c5", and a "Compare" button.

Releases Tags

Mar 11

Mrzator

v0.18.0

817d8c5

Compare

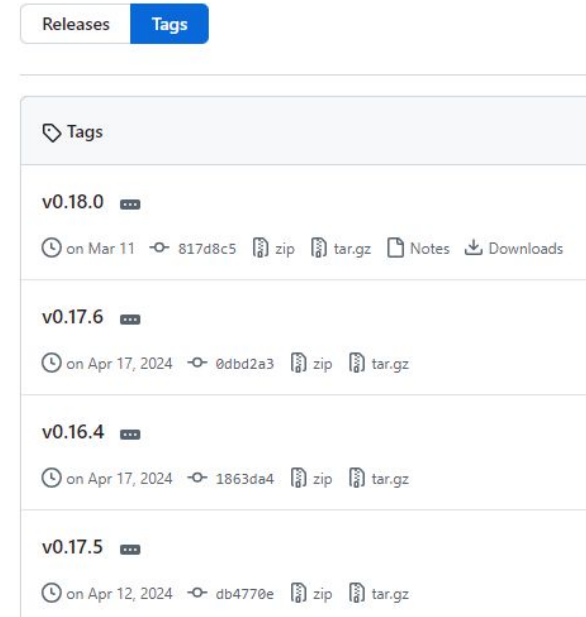
v0.18.0 (2025-03-11) Latest

Excalidraw Library

0.18.0 (2025-03-11)

Highlights

- Command palette [#7804](#)
- Multiplayer undo / redo [#7348](#)



The screenshot shows the GitHub Tags page for the repository `excalidraw/excalidraw`. The "Tags" tab is selected. It displays a list of tags: `v0.18.0`, `v0.17.6`, `v0.16.4`, and `v0.17.5`. Each tag entry includes the release date, commit hash, and download links for zip and tar.gz files. For example, `v0.18.0` was released on Mar 11 with commit hash 817d8c5.

Releases Tags

Tags

v0.18.0

on Mar 11 817d8c5 zip tar.gz Notes Downloads

v0.17.6

on Apr 17, 2024 0dbd2a3 zip tar.gz

v0.16.4

on Apr 17, 2024 1863da4 zip tar.gz

v0.17.5

on Apr 12, 2024 db4770e zip tar.gz

Dúvidas?
Mandem por Discord!



Até a próxima aula!

Lana Mesquita

